Training

clarification

- all the tensors are 3-Dimensional, according to kitti_train.py

  since they use TensorFlow, the 3 dimensions are Height, Width, and Channel.

  This is default for TensorFlow, and it is also specified in kitti_train.py

  All the conv kernels are 3*3(*Channels), which is defined in the code kitti_train.py and prednet.py

  (the input is 5-D tensor, batch size*timestep*H*W*C, but batches are trained parallel just to train faster, and different timestep is processed in sequence, so it is 3-D for all A, Ahat, R and E)

  Conv() function does not change the length or width (using padding) and does not change the channel(using the same number of kernels as the number of input tensors' channel)

  ConvLSTM() does not change the length or width but does changes the channel number to keep the consistency in layers.

  MaxPool() shrinks the width and length by half but keeps the number of channels.

The simplification of the model

Assuming that the input training set is a set of images, that has only 2*2 pixels and 2 channels, which can be translated to a 2*2*2 tensor.

Assuming that at every timestep n, the input tensor's entries are all n. e.g. at timestep 1, the tensor is an all 1 tensor.

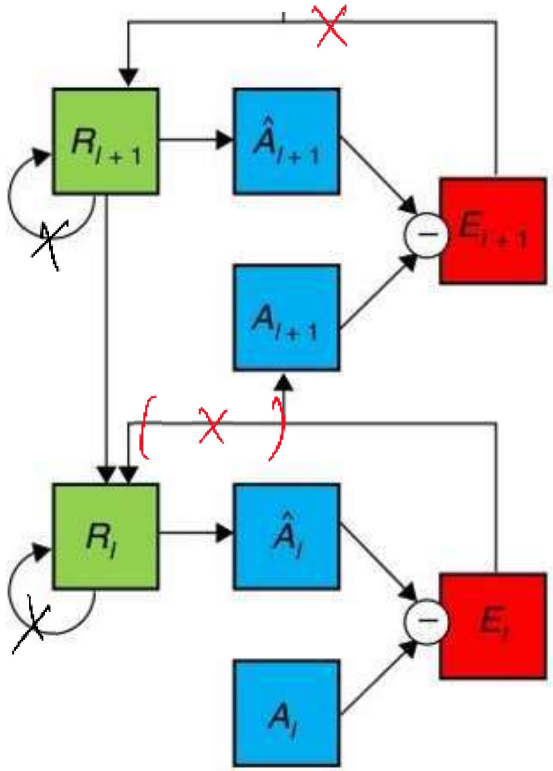
Assuming that the model has only 2 layers, layer 0 and layer 1.

The first timestep is assumed to be 1 for simplification.

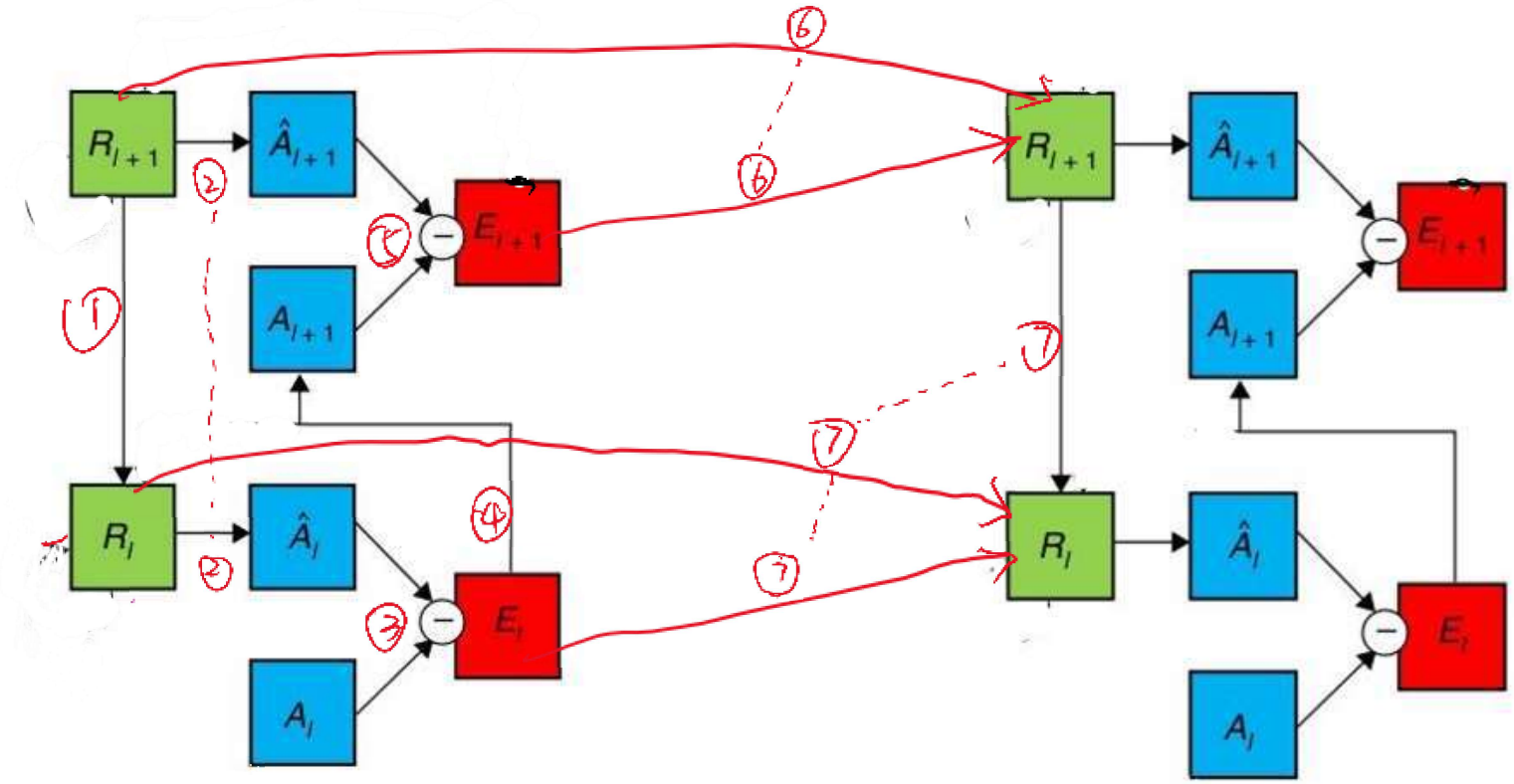
Assuming that the before training starts, initialization of all tensors are 0 tensors.

Assuming that the Conv kernels have kx+b entries, where the k is 1, and b is 0 when we start.

the original image is not clear



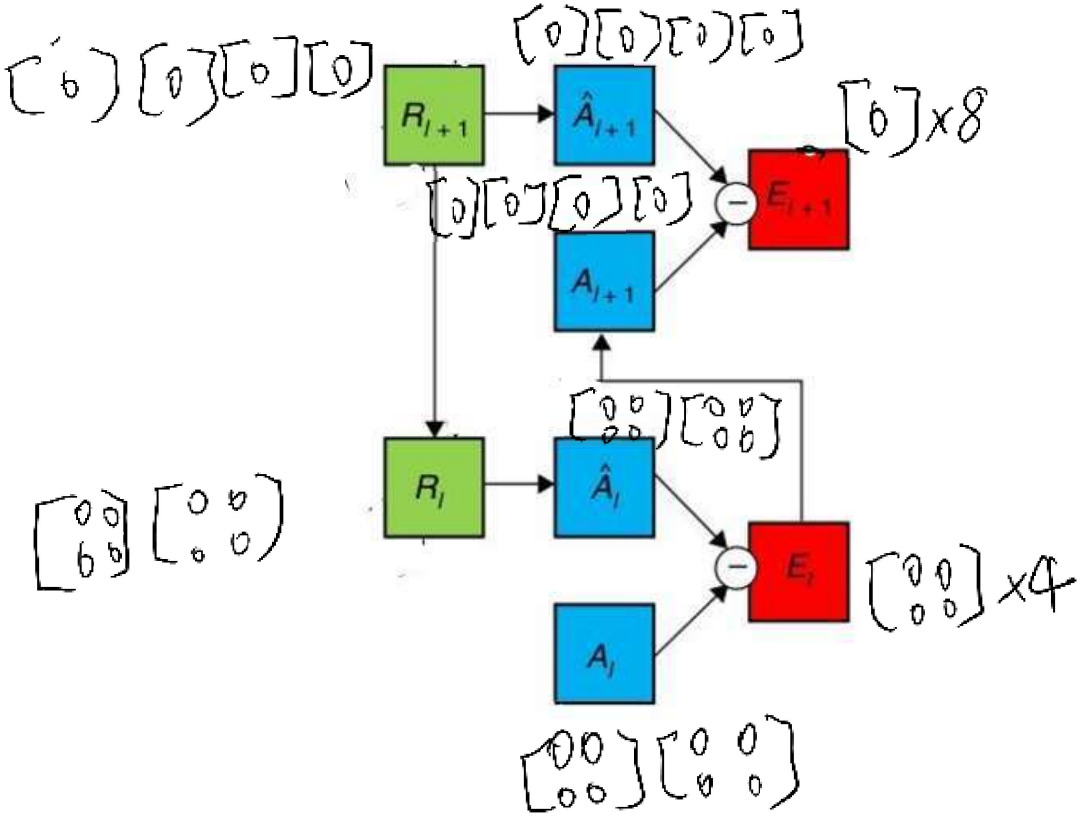
A more accurate illustration



time step 1

time step 2

intialization (before training)



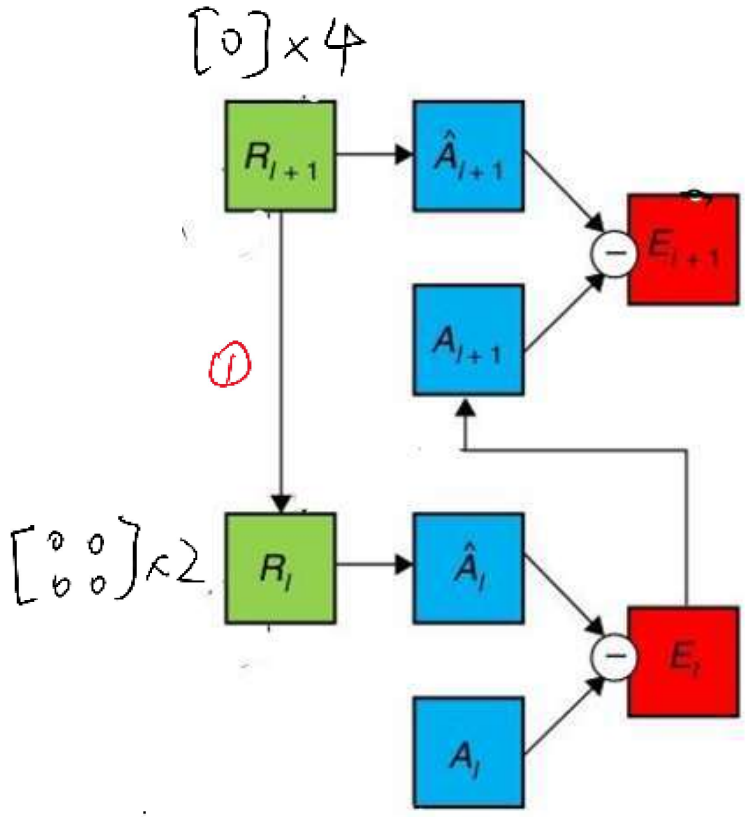
step 1

Et-1 and Rt-1 refers to the last time step (t-1). but since this is the first timestep, there is no last step, so these 2 are 0 for the moment

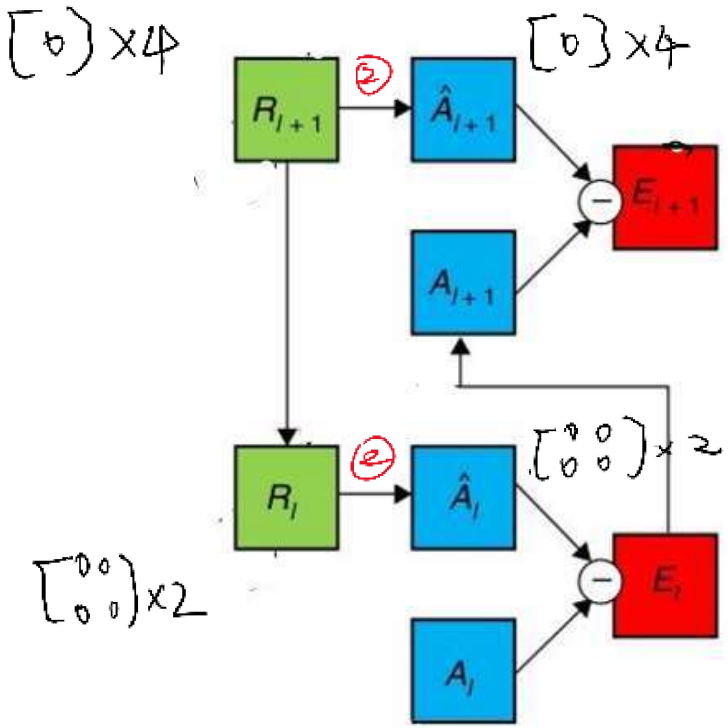
$$R_l^t = ConvLSTM\left(E_l^{t-1}, R_l^{t-1}, Upsample\left(R_{l+1}^t\right)\right)$$

(in our example, the Conv kernel is 2 since Ahat should have 2 channels)

the initialization of all parameter is 0, so Rl+1 is still 0 for the moment as well.



step 2



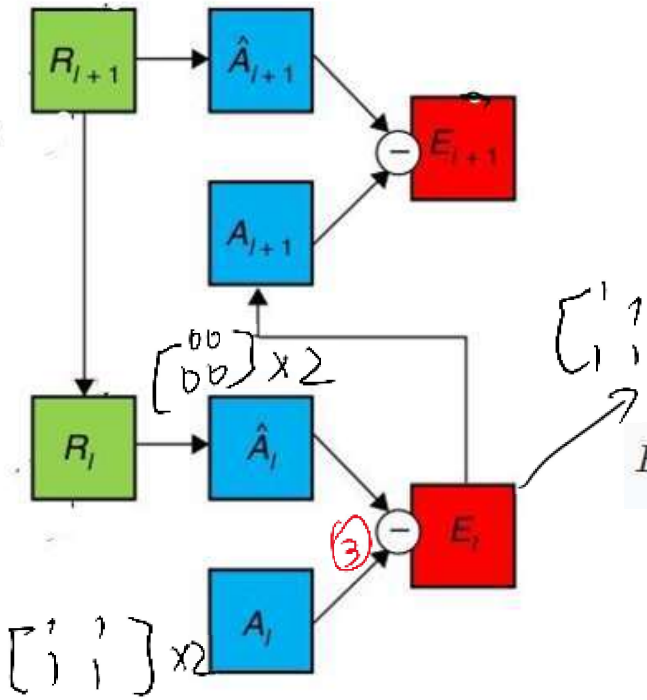
4 kernels for layer 1

2 for layer 0

↓

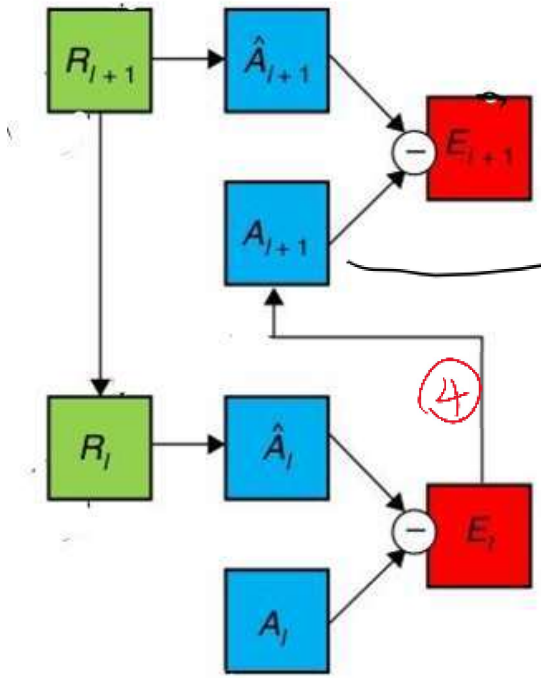
$\hat{A}_l^t = \mathrm{ReLU}(\mathrm{Conv}(R_l^t))$

step 3



$$E_l^t = \left[ReLU\left(A_l^t - \hat{A}_l^t\right); ReLU\left(\hat{A}_l^t - A_l^t\right) \right]$$

step 4



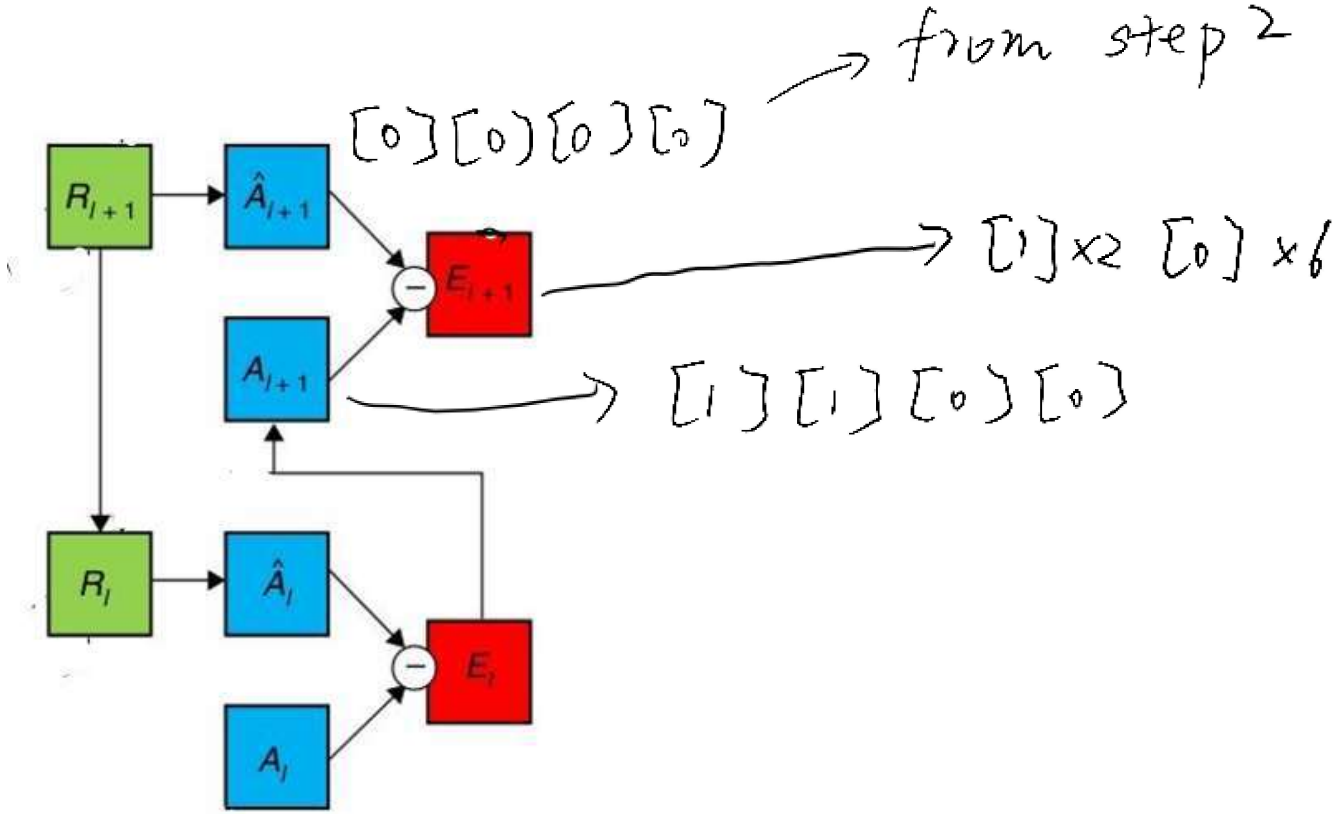
4 kernels

$$A_l^t = MaxPool\left(ReLU\left(Conv\left(E_{l-1}^t\right)\right)\right)$$

[1] [1] [0] [0]

[1 1; 1 1] [1 1; 1 1] [0 0; 0 0] [0 0; 0 0]

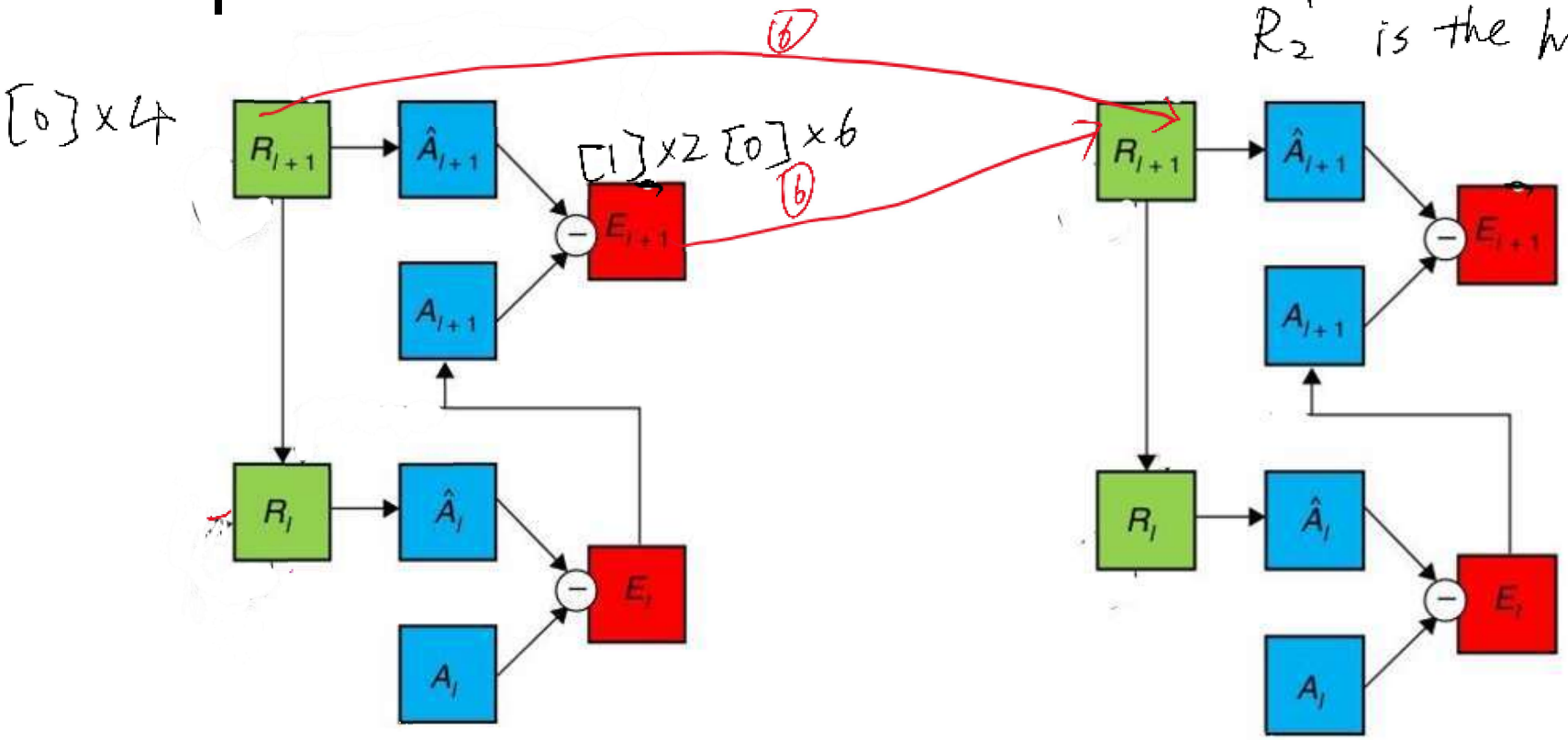
step 5



step 6

$$R_l^t = ConvLSTM\left(E_l^{t-1}, R_l^{t-1}, \cancel{Upsample}\left(R_{l+1}^t\right)\right)$$

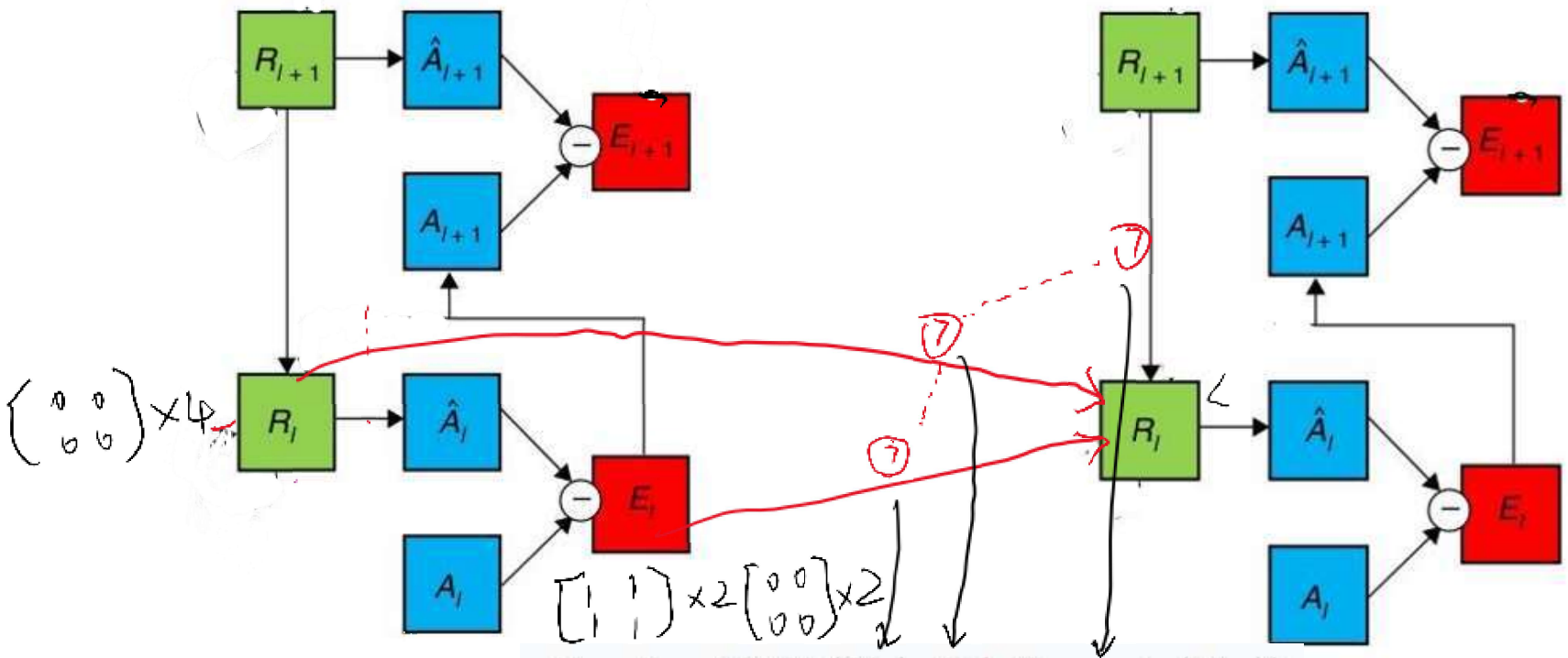
R_2 is the highest layer



time step 1

time step 2

step 7: ready for the next time step



$$R_l^t = ConvLSTM\left(E_l^{t-1}, R_l^{t-1}, Upsample\left(R_{l+1}^t\right)\right)$$

time step 1

time step 2

This cycle goes on

Q1: what does LSTM do?

- LSTM has input(i)/forget(f)/output(o) 3 gates + 1 cell states(c). They are not the parameters we want to train, but rather the memory during a single task (initialized to 0 before any task)
- they are 3-dimensional tensors just as R.
  - H*W*channel
- these 4 values updates by themselves
- i controls the ratio of the input parameters to the cell state
  - e.g. R_t-1:E_t-1:R_l+1=1:2:1
- R=o * activation_function(c)
  - in code activation is tanh
- the example before did not use the exact ConvLSTM, to simply the demonstration

a. forget gate

determines which information to discard from the cell state

$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] + b_f\right)$$

W: weight
b: bias
h: hidden status (from the last time step)
x: current input
t: time step

b. input gate

decides which new information is stored in the cell state

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$
$$\tilde{C}_t = tanh(W_C \cdot [h_{t-1}, x_t] + b_C), the\ candidate\ vectors$$

c. output gate

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$
$$h_t = o_t \circ tanh(C_t)$$

d. cell state

the core for LSTM's long-term memory.
acts like a conveyor. Information flows on it with only minor modifications.

$$C_t = f_t \circ C_{t-1} + i_t \circ \tilde{C}_t$$

Q2: how was the model trained during the process?

- in extrap_loss.py
- we take the Ahat as an example, given by the paper:

$\hat{A}_0 = W_0 * R_0$, W is the kernel for Ahat

suppose we have Loss (L)

$L = \frac{1}{2}\left|\left|A - \hat{A}\right|\right|$, E as in Error

$\frac{\partial L}{\partial \hat{A}} = \frac{1}{2N} sign(\hat{A} - A)$

thus, according to the chain law

$\frac{\partial L}{\partial W_0} = \frac{\partial L}{\partial \hat{A}_0} * R_0$

where we apply the gradient descent method.

extra 1

- the code states that, every test and trained image is 128*160 pixel and 3 channels.

  in kitti_process.py

  ```
  desired_im_sz = (128, 160)
  ```

  in kitti_train.py

  ```
  n_channels, im_height, im_width = (3, 128, 160)
  ```

- In the real model, the initialization of kernels follow Xavier initialization instead of 0.